6. Write the python program for vaccum cleanser problem

main.py Visualize Run

```python
def vacuum_cleaner():
    rooms = {'A': 'Dirty', 'B': 'Dirty'}

    for room in rooms:
        print("Vacuum is in Room", room)

        if rooms[room] == 'Dirty':
            print("Cleaning Room", room)
            rooms[room] = 'Clean'

    print("Final Status:", rooms)

vacuum_cleaner()
```

Output

```
Vacuum is in Room A
Cleaning Room A
Vacuum is in Room B
Cleaning Room B
Final Status: {'A': 'Clean', 'B': 'Clean'}
```

7.Write the python program to implement BFS.

main.py Visualize Run

```python
from collections import deque
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

def bfs(start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()

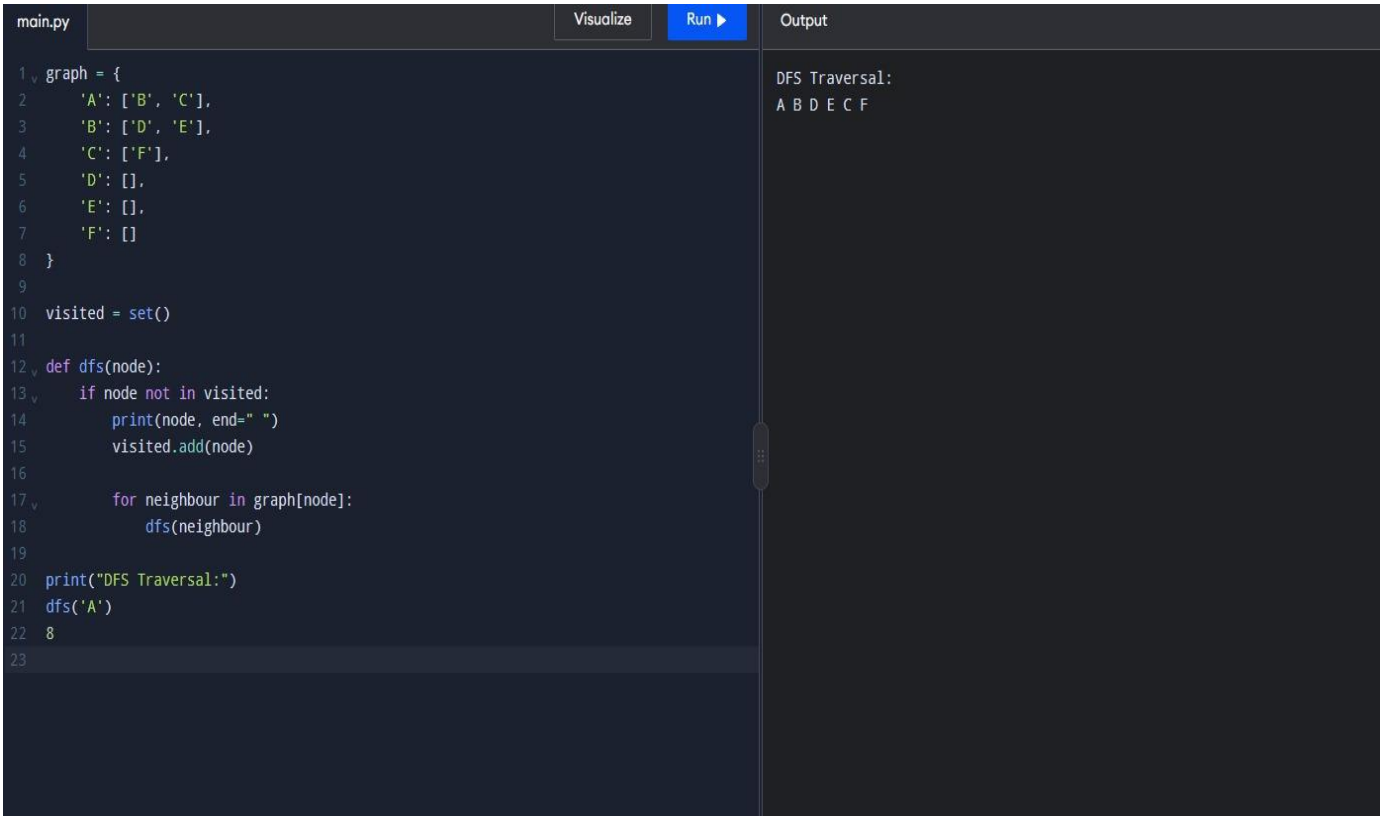
        if node not in visited:
            print(node, end=" ")
            visited.add(node)
            queue.extend(graph[node])

print("BFS Traversal:")
bfs('A')
```

Output

```
BFS Traversal:
A B C D E F
```

8. Write the python program to implement DFS.



```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = set()

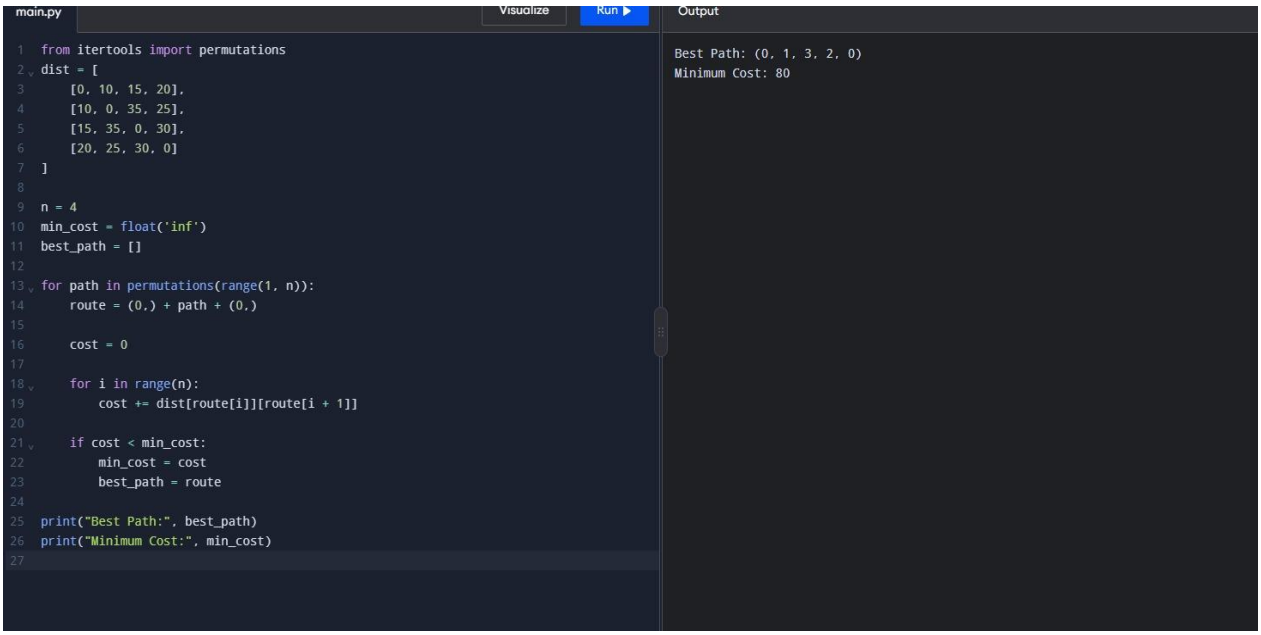
def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            dfs(neighbour)

print("DFS Traversal:")
dfs('A')
8
```

```
DFS Traversal:
A B D E C F
```

9. Write the python to implement travelling salesman problem .



```
from itertools import permutations
dist = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = 4
min_cost = float('inf')
best_path = []

for path in permutations(range(1, n)):
    route = (0,) + path + (0,)

    cost = 0

    for i in range(n):
        cost += dist[route[i]][route[i + 1]]

    if cost < min_cost:
        min_cost = cost
        best_path = route

print("Best Path:", best_path)
print("Minimum Cost:", min_cost)
```

```
Best Path: (0, 1, 3, 2, 0)
Minimum Cost: 80
```

10. write the python program to implement A* algorithm .

main.py Visualize Run

```python
import heapq
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('E', 1)],
    'D': [('F', 3)],
    'E': [('F', 2)],
    'F': []
}

h = {
    'A': 5,
    'B': 4,
    'C': 2,
    'D': 3,
    'E': 1,
    'F': 0
}

def astar(start, goal):
    queue = [(h[start], 0, start, [start])]
    best_cost = {start: 0}

    while queue:
        f, cost, node, path = heapq.heappop(queue)

        if cost > best_cost[node]:
            continue

        if node == goal:
            return path, cost

```

Output

```
Shortest Path: ['A', 'B', 'D', 'F']
Total Cost: 6
```

main.py Visualize Run ▶

```
    queue = [(h[start], 0, start, [start])]
    best_cost = {start: 0}

    while queue:
        f, cost, node, path = heapq.heappop(queue)

        if cost > best_cost[node]:
            continue

        if node == goal:
            return path, cost

        for neighbour, weight in graph[node]:
            new_cost = cost + weight

            if new_cost < best_cost.get(neighbour, float('inf')):
                best_cost[neighbour] = new_cost

                heapq.heappush(
                    queue,
                    (new_cost + h[neighbour],
                     new_cost,
                     neighbour,
                     path + [neighbour])
                )

    return None, float('inf')

path, cost = astar('A', 'F')

print("Shortest Path:", path)
print("Total Cost:", cost)
```